3. Tensors in Deep Learning

"Tensors: The Backbone of Deep Learning."

Previous Section:

2. Deep Neural Networks

Contents:

1. What is a Tensor?
    - Key Concepts of Tensors
    - The MNIST Dataset — A Real Tensor Example
    - Examples of Tensors in Real Life
    - Are Tensors Just Matrices?
2. Operations on Tensor
    - Creating Tensors with TensorFlow
    - Common Tensor Operations
    - Tensor Slicing
    - Statistical Operations on Tensors
    - Tensor Operations on MNIST

Summary

References

At first glance, Neural Networks look like they work with matrices. Inputs become tables of numbers, hidden layers perform matrix multiplications, and outputs are produced through mathematical operations.

But in Deep Learning, we usually use a broader term instead: **Tensors.** And honestly, this is one of those words that sounds more intimidating than it really

is.

We already know some basic forms of data:

- A single number → Scalar → 0D Array
- A list of numbers → Vector → 1D Array
- A table of numbers → Matrix → 2D Array

But what happens when data goes beyond two dimensions? What do we call something like:

- A stack of matrices?
- A sequence of images?
- A batch of RGB images?
- Or data with four, five, or even ten dimensions?

That is where the term **Tensor** comes in. A tensor is simply an **N-dimensional array**. So instead of inventing a new name for every possible dimension, Deep Learning uses one unified concept:

- Scalars are 0D tensors
- Vectors are 1D tensors
- Matrices are 2D tensors
- Cubes of numbers are 3D tensors
- And anything beyond that is still... a tensor

So tensors are not separate from matrices. Instead: A matrix is just one specific type of tensor. And this becomes extremely important in Deep Learning because real-world data is rarely only two-dimensional.

An image alone already contains multiple dimensions: Height; Width; and Color channels.

And once we process multiple images at once, another dimension appears: the batch dimension. So Deep Learning frameworks like TensorFlow and PyTorch don’t think in terms of “just matrices”. They think in tensors because tensors can represent data of almost any shape.

You can think of tensors as the universal language of Deep Learning. Everything becomes tensors: Inputs; Images; Audio; Text embeddings; Hidden layer outputs; Weights inside neural networks.

Even the activations flowing through a Neural Network are tensors moving from one layer to another. So when people say: ***"Deep Learning works with tensors".***

What they really mean is: Deep Learning works with multi-dimensional numerical representations of data. And once you understand tensors, Neural Networks suddenly stop feeling magical. Because underneath all the intelligence, all the predictions, and all the fancy AI terminology. It's really just tensors being transformed through layers.

1. What is a Tensor?

Here is something interesting. One of the biggest Deep Learning libraries is called TensorFlow. Notice the word **Tensor.** That is not accidental. Because tensors are the fundamental data structure of Deep Learning.

Everything inside a Neural Network eventually becomes tensors: from inputs, weights, activations, gradients, to images, videos, and audios

In many ways, tensors are to Deep Learning what matrices are to Linear Algebra. But tensors go beyond matrices. A tensor is simply a **multi-dimensional array** capable of storing data in different dimensions. That means tensors can represent scalars, vectors, matrices, and higher-dimensional structures

So instead of inventing separate names for every dimension, Deep Learning unifies everything under one concept: Tensor.

Key Concepts of Tensors

- **Rank (Dimensions):** The rank of a tensor refers to the number of axes (dimensions) it has. Examples:
  - Scalar → Rank 0 Tensor
  - Vector → Rank 1 Tensor
  - Matrix → Rank 2 Tensor
  - 3D cube of numbers → Rank 3 Tensor
  - Anything higher → Higher-order Tensor

You can visualize it like this: 0D is a point, 1D is a line, 2D is a table, 3D is a cube or stack, N-D is data beyond human-friendly visualization

- **Shape:** The shape tells us the size of each dimension. For example: $(2, 3)$ means 2 rows and 3 columns. A tensor with shape $(60000, 28, 28)$ means 60,000 samples and each image is 28 × 28 pixels
- **Data Type (dtype):** Tensors also store the type of values they contain. Examples are ***uint8***, ***float32***, and ***float64***

Deep Learning often prefers optimized numerical types because Neural Networks process enormous amounts of data. For example: Images commonly use `uint8`; Neural Network calculations often use `float32`

- **Axis:** An axis refers to a specific dimension of the tensor. Consider this tensor

```
x = [[[1, 2], [3, 4], [5, 6]], [[7, 8], [9, 10], [11, 12]]]
```

This tensor has: **Rank = 3** and **Shape = (2, 3, 2)**, Now let's understand what each axis means.

axis_0 → **The Outside Axis.** This is the outermost level.

```
[
    [...],
    [...]
]
```

There are 2 large groups. So: $\text{axis}_0 = 2$. You can think of this as samples, batches, and stacks, depending on the context.

axis_1 → **The Middle Axis.** Inside each outer group, there are 3 inner lists:

```
[
 [1,2],
 [3,4],
 [5,6]
]
```

So: $\text{axis}_1 = 3$. This often represents rows, timesteps, and height, depending on the data.

axis_2 → **The Inside Axis.** Finally, each smallest list contains 2 numbers:

```
[1, 2]
```

So: $\text{axis}_2 = 2$. This can represent columns, features, and channels, depending on the context.

The MNIST Dataset — A Real Tensor Example

One of the most famous datasets in Deep Learning is the MNIST handwritten digit dataset.

```
# Load the MNIST dataset
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) =
mnist.load_data()

# Show the rank, shape and dtype
print("Rank:", train_images.ndim)
print("Shape:", train_images.shape)
print("Data Type:", train_images.dtype)
```

Rank: 3
Shape: (60000, 28, 28)
Data Type: uint8

This means:

- Rank 3 → A 3D Tensor
- 60,000 images
- Each image is 28 × 28 pixels
- Pixel values are stored as unsigned integers

You can interpret the shape like this: `(samples,height,width)` or `(axis_1, axis_2, axis_3)`

So MNIST is essentially a stack of images stored together inside one large tensor.

Examples of Tensors in Real Life

- **Vector Data:** Customer Information with Age, Salary, and Credit score. Shape: `(samples,features)` → This is usually a 2D tensor.
- **Time Series / Sequential Data:** Stock prices, Sensor readings, Weather data. Shape: `(samples,timesteps,features)` → This becomes a 3D tensor.
- **Image Data:** For grayscale or RGB images, the shape is `(samples,height,width,channels)` → grayscale image has 1 channel, RGB image has 3 channels
- **Video Data:** Videos add another dimension which are frames → Shape: `(samples,frames,height,width,channels)`. This becomes a very high-dimensional tensor.

Are Tensors Just Matrices?

Not exactly. A matrix is specifically a 2D Tensor. So tensors are not matrices. But matrices are tensors. Similarly: Scalars are tensors, Vectors are tensors, Matrices are tensors, Higher-dimensional arrays are tensors.

So tensors are the general concept that includes all of them. And this is why Deep Learning relies so heavily on tensors. Because real-world data is rarely only two-dimensional.

2. Operations on Tensor

Now that we understand what tensors are, the next question becomes: “What do we actually do with them?”

Because in Deep Learning, tensors are not just containers for data. They are constantly being transformed, sliced, multiplied, reshaped, and passed through

Neural Networks. And honestly, most Deep Learning libraries are really just giant tensor manipulation systems.

In this section, we'll focus mainly on: 1D tensors, 2D tensors, and 3D tensors. Especially the MNIST dataset, since it is one of the easiest ways to visualize tensor operations.

One important note before we begin: TensorFlow tensors are very similar to NumPy arrays. In fact, most of the time, we first create data using NumPy, then convert it into tensors for Deep Learning operations. So let's start there.

Creating Tensors with TensorFlow

First, import the libraries:

```
import numpy as np
import tensorflow as tf
```

Now let's create some arrays.

- **1D Tensor**

```
array1 = np.array([1, 2, 3, 4, 5])

tensor1 = tf.convert_to_tensor(array1)

print("1D Tensor")
print("Rank:", tensor1.ndim)
print("Shape:", tensor1.shape)
```

```
1D Tensor
Rank: 1
Shape: (5,)
```

This is simply a vector.

- **2D Tensor**

```
array2 = np.array([
    [10, 20, 30, 40],
    [50, 60, 70, 80],
    [90, 100, 110, 120]
])

tensor2 = tf.convert_to_tensor(array2)

print("2D Tensor")
print("Rank:", tensor2.ndim)
print("Shape:", tensor2.shape)
```

```
2D Tensor
Rank: 2
Shape: (3, 4)
```

You can interpret this as: `(rows,columns)`. So this tensor contains 3 rows and 4 columns

- **3D Tensor**

```
array3 = np.array([
    [
        [1, 2],
        [3, 4],
        [5, 6]
    ],

    [
        [7, 8],
        [9, 10],
        [11, 12]
    ]
])

tensor3 = tf.convert_to_tensor(array3)

print("3D Tensor")
print("Rank:", tensor3.ndim)
print("Shape:", tensor3.shape)
```

```
3D Tensor
Rank: 3
Shape: (2, 3, 2)
```

You can interpret the shape like this: `(axis_0,axis_1,axis_2)` . Or more visually: 2 groups, each group has 3 rows, each row contains 2 values

Common Tensor Operations

Deep Learning constantly performs mathematical operations on tensors. The most common ones are: Addition, Subtraction, Multiplication, and Dot Product

- **Addition**

```
a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])

print(a + b)
```

tf.Tensor([5 7 9], shape=(3,), dtype=int32)

TensorFlow performs element-wise addition.

- **Subtraction**

```
print(a - b)
```

tf.Tensor([-3 -3 -3], shape=(3,), dtype=int32)

- **Multiplication**

```
print(a * b)
```

tf.Tensor([4 10 18], shape=(3,), dtype=int32)

Again, this is element-wise multiplication.

- **Dot Product**

The dot product is one of the most important operations in Neural Networks. Because every neuron essentially performs where: weights are multiplied with inputs and then summed together

```
x = tf.constant([[1, 2]])
w = tf.constant([[3], [4]])

result = tf.matmul(x, w)

print(result)
```

tf.Tensor([[11]], shape=(1, 1), dtype=int32)

Because: $(1 \times 3) + (2 \times 4) = 11$. This operation is everywhere in Deep Learning.

Tensor Slicing

One powerful feature of tensors is slicing. This allows us to extract rows, columns, specific values, sections of data. All are important for preprocessing datasets.

- **Get a Specific Value**

```
print(array2[1, 2])
```

70

Meaning row index 1 and column index 2

- **Get an Entire Row**

```
print(array2[0])
```

[10 20 30 40]

- **Get an Entire Column**

```
print(array2[:, 1])
```

```
[ 20  60 100]
```

The colon means: “Select everything along this axis.”

- **Slicing a 3D Tensor**

```
print(array3[0])
```

```
[[1 2]
 [3 4]
 [5 6]]
```

This selects the first major block of the tensor.

Statistical Operations on Tensors

Deep Learning also performs statistical operations constantly. Especially during normalization, feature scaling, and optimization

- **Mean**

```
print(tf.reduce_mean(tensor2))
```

This computes the average of the entire tensor.

- **Mean Along Columns**

```
print(tf.reduce_mean(tensor2, axis=0))
```

This computes the mean column-wise.

- **Mean Along Rows**

```
print(tf.reduce_mean(tensor2, axis=1))
```

This computes the mean row-wise.

- **Minimum and Maximum**

```
print(tf.reduce_min(tensor2))
print(tf.reduce_max(tensor2))
```

These operations are heavily used in data preprocessing.

Tensor Operations on MNIST

Now let's connect everything back to Deep Learning.

```
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) =
mnist.load_data()
print(train_images.shape)
```

(60000, 28, 28)

This means: `(samples,height,width)` → So: 60,000 images, each image is 28 × 28 pixels

- Now let's slice the first image:

```
digit = train_images[0]
print(digit.shape)
```

(28, 28)

This extracts one single handwritten digit image from the dataset.

- You can also slice smaller regions:

```
my_slice = train_images[0, 10:20, 10:20]

print(my_slice.shape)
```

```
(10, 10)
```

This extracts: rows 10 → 20 and columns 10 → 20 from the image.

Summary

Tensor operations are the foundation of Deep Learning. Every Neural Network is essentially doing this repeatedly: transforming tensors, multiplying tensors, slicing tensors, updating tensors

So underneath all the “AI magic”, Deep Learning is really just: Mathematical operations performed on tensors through layers.

References

https://www.geeksforgeeks.org/deep-learning/what-is-tensor-and-tensor-shapes/

http://youtube.com/watch?v=IMG5O6cezKA

https://www.youtube.com/watch?v=L35fFDpwIM4

Next Section:

4. Neural Networks: The Nine-Step Workflow